

CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 3: Pointers, Arrays, and C Memory Model

Instructor: Justin Yokota

Agenda

- Pointers
 - Pointers and Functions
- Arrays
- Strings
- C Memory model
- Stack
- Heap

Agenda

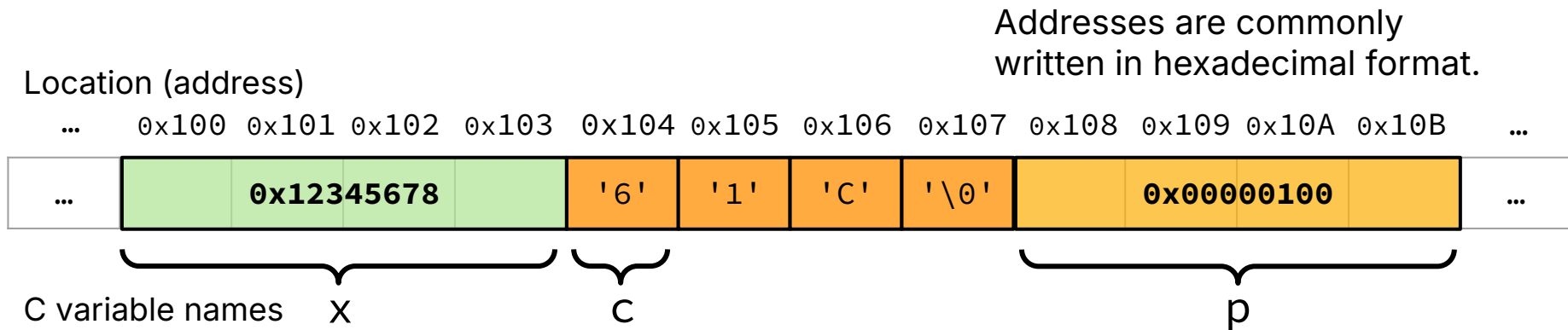
- **Pointers**
 - Pointers and Functions
- Arrays
- Strings
- C Memory model
- Stack
- Heap

Units of memory

- A **bit** is a single **binary** digit, corresponding to a single 1 or 0
- A **byte** is a collection of 8 bits. Bytes are often the smallest unit of memory that gets used for practical purposes.
 - Ex. A single char is 1 byte
- A **nibble** (or **nybble**) is a collection of 4 bits. Mostly used when talking about parts of bytes (because 1 nibble is 1 hex digit)
- Binary SI prefixes are used to describe larger memory sizes
 - Ex. 1 KiB = 1024 Bytes
- Most modern systems are 32-bit or 64-bit systems. For now, this is the largest block of memory that the CPU works with at a time.
 - 32-bit systems prefer `int32_ts` for this reason
 - Most of this class will be working with 32-bit architectures
- For this class, a **word** is a block of 4 bytes (32 bits). Different sources have different definitions on what a word is...

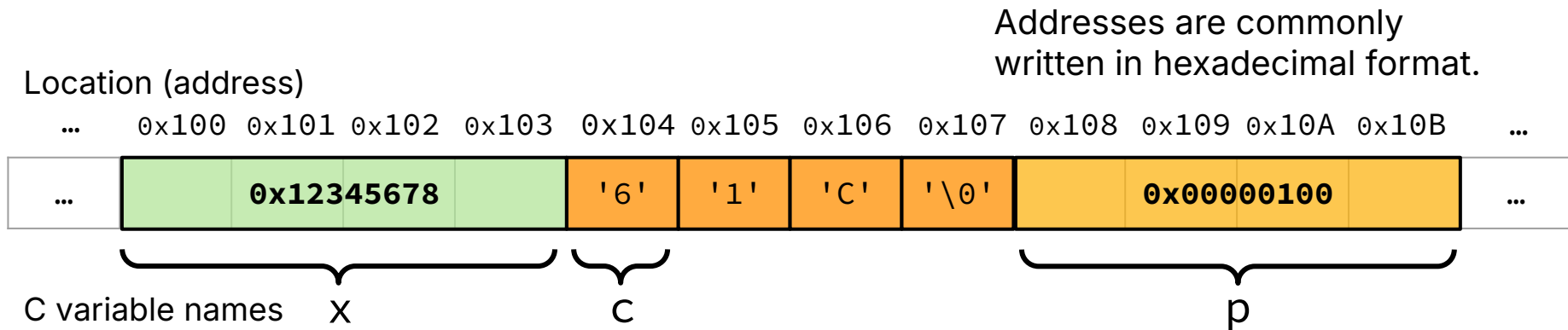
Memory is a Single Huge Array

- The memory of a program acts like a really big array of bytes
 - Each cell of the array has an **address** associated with it.
 - Ex. The 256th byte of memory is located at address 0x100
 - For now, we can imagine we have 2^{32} bytes of memory; one byte for each possible 32-bit address



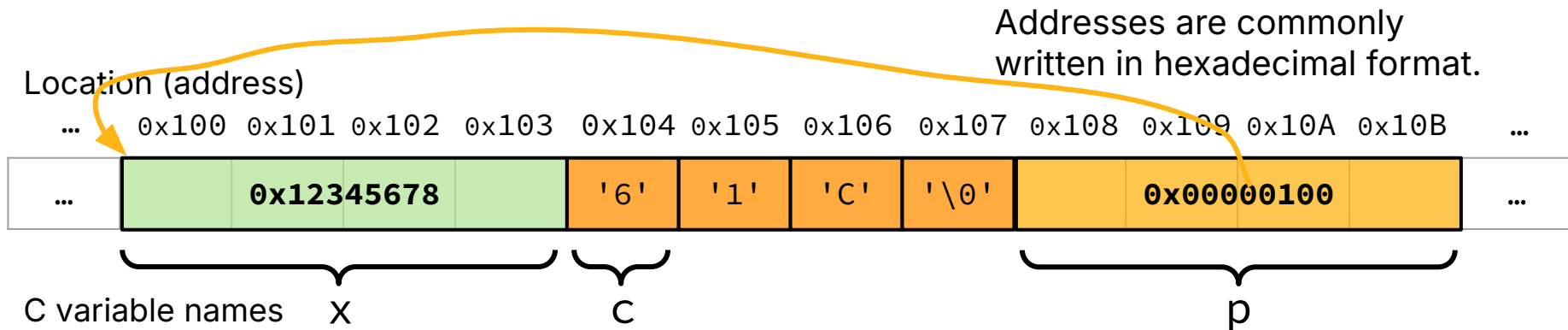
Memory is a Single Huge Array

- Each cell stores one byte of data.
- Adjacent cells can "work together" to store larger values.
 - We'll discuss tomorrow how data gets split into individual bytes, but as long as reading and writing is consistent, the exact process doesn't matter.
- Even if a value takes up multiple addresses, we say that the value is located at its lowest memory address.
 - Ex. Variable `x` is stored in addresses `0x100` through `0x103`, and so is "located" at address `0x100`.



Pointers store addresses

- Pointer: A variable that contains the memory address of another variable
 - In other words, it "points" to a memory location
- Remember, the raw binary has no meaning by itself; we can choose to interpret any chunk of 32 bits as an integer/pointer (though in most cases that yields a random/meaningless value).



Pointers in C

- To declare a pointer, we add a `*` to the pointed type
- The `*` can be either next to the variable name, or the type name
 - Technically only one of these is correct (more on this soon)
- All pointers are the same size (regardless of the type they point to): 4 bytes in a system with a 32-bit address space

		+0	+1	+2	+3
<code>int x</code>	0x100	0x12345678			
<code>char c</code>	0x104		'1'		
<code>int *r</code>	0x108	0x00000100			
<code>int ****u</code>	0x10C	0x00000114			
<code>int* p</code>	0x110	0x00000100			
<code>int ***t</code>	0x114	0x0000011C			
<code>char *q</code>	0x118	0x00000105			
<code>int **s</code>	0x11C	0x00000110			
	...				

C Pointer Example

```
1  int *p;
2  int x = 3;
3  p = &x;
4  printf("Value of *p is %d\n", *p);
5  *p = 5;
6  printf("Value of x is %d\n", x);
7  p = 16;
8  printf("Value of *p is %d\n", *p);
```

- When we start a C function, we assign space for all declared variables before running anything (think 61A environment diagrams)
 - All variables initially garbage

		+0	+1	+2	+3
int *p	0x100		????????		
int x	0x104		????????		
	0x108				
	0x10C				
	0x110				
	0x114				
	0x118				
	0x11C				
	...				

C Pointer Example

```
1 int *p;  
2 int x = 3;  
3 p = &x;  
4 printf("Value of *p is %d\n", *p);  
5 *p = 5;  
6 printf("Value of x is %d\n", x);  
7 p = 16;  
8 printf("Value of *p is %d\n", *p);
```

- Tells compiler that p is (or will be) the address of an int

		+0	+1	+2	+3
int *p	0x100		????????		
int x	0x104		????????		
	0x108				
	0x10C				
	0x110				
	0x114				
	0x118				
	0x11C				
	...				

C Pointer Example

```
1  int *p;  
2  int x = 3;  
3  p = &x;  
4  printf("Value of *p is %d\n", *p);  
5  *p = 5;  
6  printf("Value of x is %d\n", x);  
7  p = 16;  
8  printf("Value of *p is %d\n", *p);
```

- Sets x to 3

		+0	+1	+2	+3
int *p	0x100	????????			
int x	0x104	3			
	0x108				
	0x10C				
	0x110				
	0x114				
	0x118				
	0x11C				
	...				

C Pointer Example

```
1  int *p;
2  int x = 3;
3  p = &x;
4  printf("Value of *p is %d\n", *p);
5  *p = 5;
6  printf("Value of x is %d\n", x);
7  p = 16;
8  printf("Value of *p is %d\n", *p);
```

- The "&" is the **address operator**; it gets the address of a variable
- x is located at 0x104, so p gets set to 0x104

		+0	+1	+2	+3
int *p	0x100	0x00000104			
int x	0x104	3			
	0x108				
	0x10C				
	0x110				
	0x114				
	0x118				
	0x11C				
	...				

C Pointer Example

```
1  int *p;
2  int x = 3;
3  p = &x;
4  printf("Value of *p is %d\n", *p);
5  *p = 5;
6  printf("Value of x is %d\n", x);
7  p = 16;
8  printf("Value of *p is %d\n", *p);
```

- The "*" here is the **dereference operator**; it **gets** the value pointed to by p
- p is 0x104, so *p is the data at address 0x104, which is the number 3 (prints out 3)

		+0	+1	+2	+3
int *p	0x100	0x00000104			
int x	0x104	3			
	0x108				
	0x10C				
	0x110				
	0x114				
	0x118				
	0x11C				
	...				

C Pointer Example

```
1  int *p;
2  int x = 3;
3  p = &x;
4  printf("Value of *p is %d\n", *p);
5  *p = 5;
6  printf("Value of x is %d\n", x);
7  p = 16;
8  printf("Value of *p is %d\n", *p);
```

- **Changes** the value pointed to by p
- p is 0x104, so *p is the data at address 0x104. That data gets set to number 5

		+0	+1	+2	+3
int *p	0x100	0x00000104			
int x	0x104	5			
	0x108				
	0x10C				
	0x110				
	0x114				
	0x118				
	0x11C				
	...				

C Pointer Example

```
1  int *p;
2  int x = 3;
3  p = &x;
4  printf("Value of *p is %d\n", *p);
5  *p = 5;
6  printf("Value of x is %d\n", x);
7  p = 16;
8  printf("Value of *p is %d\n", *p);
```

- Looks at the value corresponding to variable x
- Currently 5, so prints 5

		+0	+1	+2	+3
int *p	0x100	0x00000104			
int x	0x104	5			
	0x108				
	0x10C				
	0x110				
	0x114				
	0x118				
	0x11C				
	...				

C Pointer Example

```
1  int *p;
2  int x = 3;
3  p = &x;
4  printf("Value of *p is %d\n", *p);
5  *p = 5;
6  printf("Value of x is %d\n", x);
7  p = 16;
8  printf("Value of *p is %d\n", *p);
```

- Sets p to 16
- Now p is **pointing** to address 16 (*p is unaffected)
- **DO NOT DO THIS**

		+0	+1	+2	+3
int *p	0x100	0x00000010			
int x	0x104	5			
	0x108				
	0x10C				
	0x110				
	0x114				
	0x118				
	0x11C				
	...				

C Pointer Example

```
1  int *p;  
2  int x = 3;  
3  p = &x;  
4  printf("Value of *p is %d\n", *p);  
5  *p = 5;  
6  printf("Value of x is %d\n", x);  
7  p = 16;  
8  printf("Value of *p is %d\n", *p);
```

- p is 0x00000010
- The print tries to access whatever data is at address 0x010, which is undefined behavior

		+0	+1	+2	+3
int *p	0x100	0x00000010			
int x	0x104	5			
	0x108				
	0x10C				
	0x110				
	0x114				
	0x118				
	0x11C				
	...				

C Pointer Example

```
1  int *p;
2  int x = 3;
3  p = &x;
4  printf("Value of *p is %d\n", *p);
5  *p = 5;
6  printf("Value of x is %d\n", x);
7  p = 16;
8  printf("Value of *p is %d\n", *p);
```

- Possibility 1: We access whatever data happens to be at location 0x010, and interpret the binary there as if it was an int (even if that data's garbage, char data, etc.)

		+0	+1	+2	+3
int *p	0x100	0x00000010			
int x	0x104	5			
	0x108				
	0x10C				
	0x110				
	0x114				
	0x118				
	0x11C				
	...				

C Pointer Example

```
1  int *p;  
2  int x = 3;  
3  p = &x;  
4  printf("Value of *p is %d\n", *p);  
5  *p = 5;  
6  printf("Value of x is %d\n", x);  
7  p = 16;  
8  printf("Value of *p is %d\n", *p);
```

- Possibility 2: The computer realizes we tried to access a memory location we weren't supposed to, and forces an error (segmentation fault)

		+0	+1	+2	+3
int *p	0x100	0x00000010			
int x	0x104	5			
	0x108				
	0x10C				
	0x110				
	0x114				
	0x118				
	0x11C				
	...				

Pointer Syntax Summary

- Pointers: *reference* another variable, stores a memory address
- * can be used 3 different ways:
 - `int *p`
 - `declares` that `p` is a pointer
 - `*p`
 - `gets the value` that `p` points to
 - `*p =`
 - `changes the value` that `p` points to
- & can be used to get the address of a variable

Don't define multiple variables in the same line!

- C lets you define pointers either with `int* p` or `int *p`
 - Technically the latter is more correct
 - But the former feels more "intuitive" from an object-oriented perspective
- Former (technically wrong): "p is a variable of type int"
- Latter: "p is a variable such that *p has type int"
- Affects things when you declare two variables in the same line
 - Don't do this. If you can make a mistake, assume you will make that mistake

```
int* p, q;    // p is an int pointer, q is an int
int *p, *q;   // p and q are int pointers.
```

Pointers in C ... The Good, the Bad, and the Ugly

- To pass a large struct or array to a function, it's **easier/faster/etc.** to pass a pointer.
 - Otherwise, we'd need to copy a huge amount of data!
- At the time C was invented (early 1970s), compilers didn't produce efficient code, so C was designed to give human programmer more flexibility.
 - Nowadays, computers are 100,000x faster; compilers are also way, way, way better.
- Still used for low-level system code, as well as implementation of “pass-by-reference” object paradigms in other languages.
- In general, pointers allow cleaner, more compact code.

Pointers in C ... The Good, the Bad, and the Ugly

! So, what are the drawbacks?

- Pointers are probably the single largest source of bugs in C.

Be careful!

- Most problematic with dynamic memory management
- Dangling references and memory leaks

} (more later)

Common C Bug

Declaring a pointer does not allocate space for something to be pointed to (this just tries to change a random memory location)

```
1  int main(int argc, char* argv[]) {  
2      int *p;  
3      *p = 5;  
4  }
```

```
$ gcc main.c  
$ ./a.out  
[1] 45130 bus error  ./a.out
```

		+0	+1	+2	+3
int *p	0x100	????????			
	0x104				
	0x108				
	0x10C				
	0x110				
	0x114				
	0x118				
	0x11C				
	...				

NULL pointers...

The pointer of all 0s is special.

- The **NULL** pointer, like in Java, Python, etc...

```
char *p = NULL;
```

p

0x00000000

If you write to or read from a null pointer, your program should **crash**.

Since **0 is false**, its very easy to do tests for null:

- `if(!p) { /* p is a null pointer */ }`
- `if(q) { /* q is not a null pointer */ }`

Agenda

- **Pointers**
 - **Pointers and Functions**
- Arrays
- Strings
- C Memory model
- Stack
- Heap

C Functions

C is **pass-by-value**: A function parameter gets assigned a **copy of the argument value**.

```
void addOne (int x)
{
    x = x + 1;
}
```

```
int y = 3;
addOne(y);
```

		+0	+1	+2	+3
main Frame int y	0xFFC				
	0xFF8		???????		
	0xFF4				
	0xFF0				
	0xFEC				
	0xFE8				
	0xFE4				
	0xFE0				
	...				

C Functions

C is **pass-by-value**: A function parameter gets assigned a **copy of the argument value**.

```
void addOne (int x)
{
    x = x + 1;
}
```

```
int y = 3;
addOne(y);
```

		+0	+1	+2	+3
main Frame int y	0xFFC				
	0xFF8		3		
	0xFF4				
	0xFF0				
	0xFEC				
	0xFE8				
	0xFE4				
	0xFE0				
	...				

C Functions

C is **pass-by-value**: A function parameter gets assigned a **copy of the argument value**.

```
void addOne (int x)
{
    x = x + 1;
}
```

```
int y = 3;
addOne(y);
```

		+0	+1	+2	+3
main Frame		0xFFC			
	int y	0xFF8		3	
		0xFF4			
addOne Frame		0xFF0			
	int x	0xFEC		3	
		0xFE8			
		0xFE4			
		0xFE0			
	...				

C Functions

C is **pass-by-value**: A function parameter gets assigned a **copy of the argument value**.

```
void addOne (int x)
{
    x = x + 1;
}
```

```
int y = 3;
addOne(y);
```

		+0	+1	+2	+3
main Frame		0xFFC			
	int y	0xFF8		3	
		0xFF4			
addOne Frame		0xFF0			
	int x	0xFEC		4	
		0xFE8			
		0xFE4			
		0xFE0			
	...				

C Functions

y doesn't change if we try to run this code

```
void addOne (int x)
{
    x = x + 1;
}
```

```
int y = 3;
addOne(y);
```

		+0	+1	+2	+3
main Frame	0xFFC				
	int y 0xFF8		3		
	0xFF4				
	0xFF0				
	0xFEC				
	0xFE8				
	0xFE4				
	0xFE0				
...					

Making addOne

How would you change the addOne function so that it properly increments y?

```
void addOne (int x)
{
    x = x + 1;
}
```

```
int y = 3;
addOne(y);
```

		+0	+1	+2	+3
main Frame int y	0xFFC				
	0xFF8		???????		
	0xFF4				
	0xFF0				
	0xFEC				
	0xFE8				
	0xFE4				
	0xFE0				
	...				

Making addOne

Send in a copy of the pointer!

```
void addOne (int *x)
{
    *x = *x + 1;
}
```

```
int y = 3;
addOne(&y);
```

		+0	+1	+2	+3
main Frame	0xFFC				
	int y 0xFF8		???????		
	0xFF4				
	0xFF0				
	0xFEC				
	0xFE8				
	0xFE4				
	0xFE0				
...					

Making addOne

Send in a copy of the pointer!

```
void addOne (int *x)
{
    *x = *x + 1;
}
```

```
int y = 3;
addOne(&y);
```

		+0	+1	+2	+3
main Frame	0xFFC				
	int y 0xFF8		3		
	0xFF4				
	0xFF0				
	0xFEC				
	0xFE8				
	0xFE4				
	0xFE0				
...					

Making addOne

Send in a copy of the pointer!

```
void addOne (int *x)
```

```
{
```

```
    *x = *x + 1;
```

```
}
```

```
int y = 3;
```

```
addOne(&y);
```

		+0	+1	+2	+3
main Frame	0xFFC				
	int y	0xFF8	3		
	0xFF4				
addOne Frame	0xFF0				
	int *x	0xFEC	0xFF8		
	0xFE8				
	0xFE4				
	0xFE0				
	...				

Making addOne

$\ast(0xFF8) = 3$

$3+1 = 4$

Set 0xFF8 to 4

```
void addOne (int *x)
```

```
{
```

```
    *x = *x + 1;
```

```
}
```

```
int y = 3;
```

```
addOne(&y);
```

		+0	+1	+2	+3
main Frame		0xFFC			
	int y	0xFF8		4	
		0xFF4			
addOne Frame		0xFF0			
	int *x	0xFEC		0xFF8	
		0xFE8			
		0xFE4			
		0xFE0			
	...				

Making addOne

y successfully set to 4

```
void addOne (int *x)
{
    *x = *x + 1;
}
```

```
int y = 3;
addOne(&y);
```

		+0	+1	+2	+3
main Frame	0xFFC				
	int y 0xFF8		4		
	0xFF4				
	0xFF0				
	0xFEC				
	0xFE8				
	0xFE4				
	0xFE0				
...					

Passing structs as function inputs

If a function is passed a struct, it may need to copy a lot of data

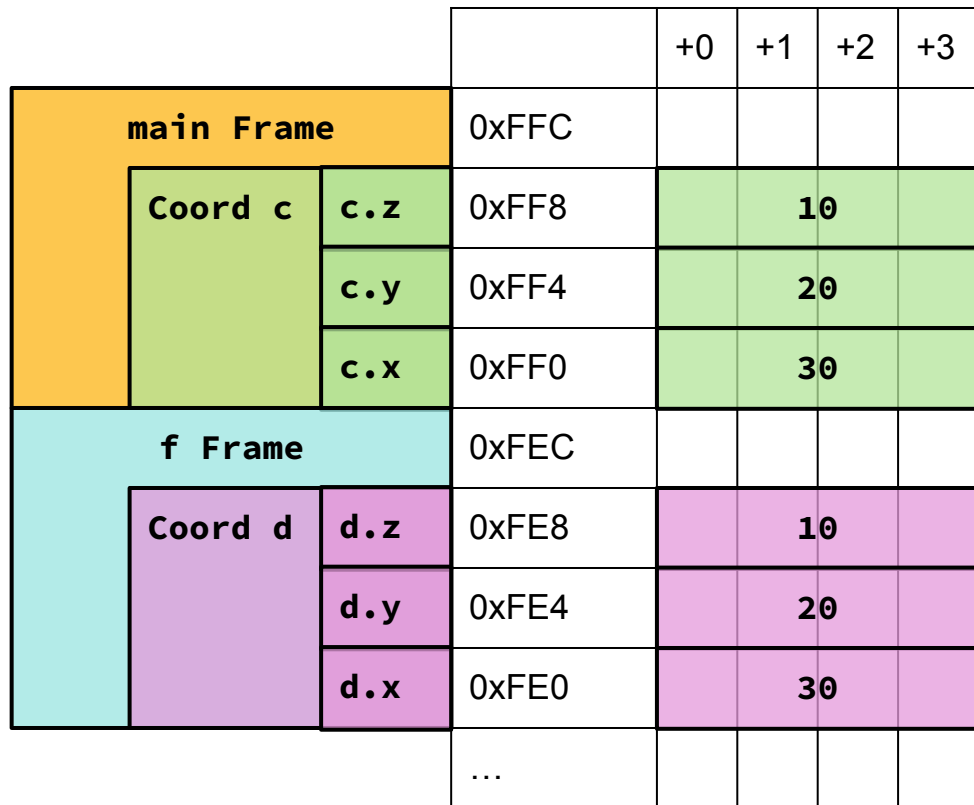
```
typedef struct {  
    int x;  
    int y;  
    int z;  
} Coord;  
int f(Coord d)  
{  
    return d.x;  
}  
Coord c;  
f(c);
```

		+0	+1	+2	+3
main Frame	Coord c				
	c.z	0xFFC			
	c.y	0xFF8			
	c.x	0xFF4			
		0xFF0			
		0xFFC			
		0xFF8			
		0xFF4			
		0xFF0			
		0xFFC			
		0xFF8			
		0xFF4			
		0xFF0			
		...			

Passing structs as function inputs

If a function is passed a struct, it may need to copy a lot of data (3 ints copied to copy c)

```
typedef struct {  
    int x;  
    int y;  
    int z;  
} Coord;  
int f(Coord d)  
{  
    return d.x;  
}  
Coord c;  
f(c);
```



Passing structs as function inputs

Better approach: pass in a pointer (always 4 bytes, regardless of struct size)

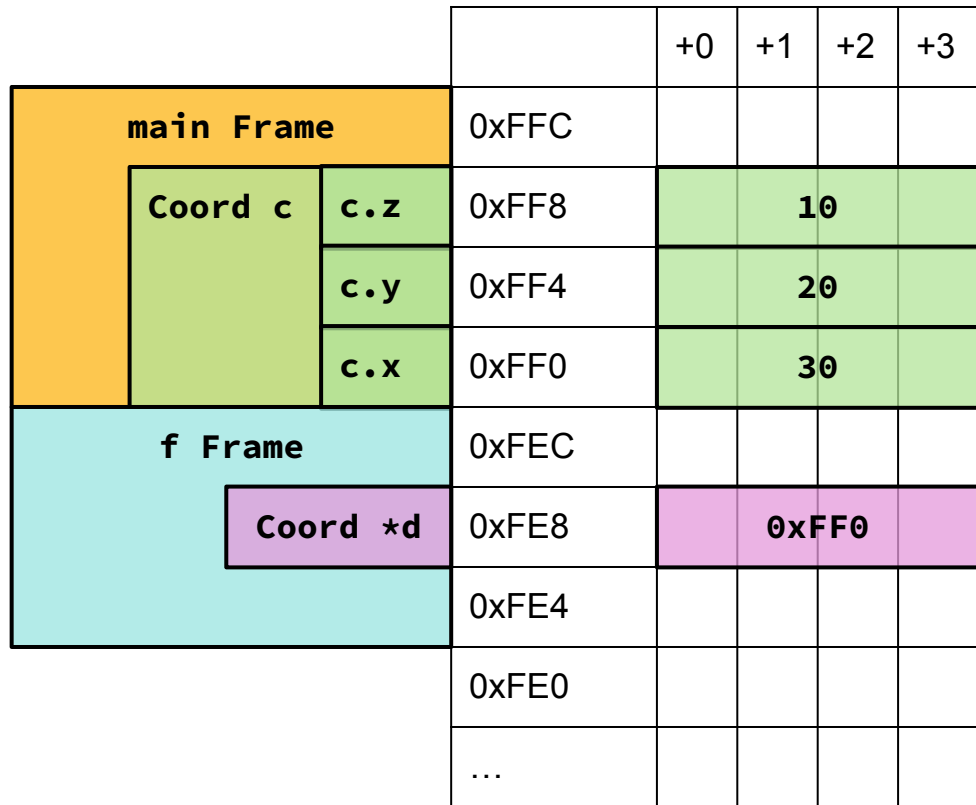
```
typedef struct {  
    int x;  
    int y;  
    int z;  
} Coord;  
int f(Coord *d)  
{  
    return (*d).x;  
}  
Coord c;  
f(&c);
```

		+0	+1	+2	+3
main Frame		0xFFC			
	Coord c	c.z	0xFF8	10	
		c.y	0xFF4	20	
		c.x	0xFF0	30	
		0xFEC			
		0xFE8			
		0xFE4			
		0xFE0			
		...			

Passing structs as function inputs

Better approach: pass in a pointer (always 4 bytes, regardless of struct size)

```
typedef struct {
    int x;
    int y;
    int z;
} Coord;
int f(Coord *d)
{
    return (*d).x;
}
Coord c;
f(&c);
```



Passing structs as function inputs

Arrow notation for structs:
Same as dereference, then
access

```
typedef struct {  
    int x;  
    int y;  
    int z;  
} Coord;  
int f(Coord *d)  
{  
    //Same as (*d).x;  
    return d->x;  
}  
Coord c;  
f(&c);
```

		+0	+1	+2	+3
main Frame	Coord c				
	c.z	0xFFC			
	c.y	0xFF8		10	
	c.x	0xFF4		20	
		0xFF0		30	
		0xFEC			
		0xFE8			
		0xFE4			
		0xFE0			
		...			

Agenda

- Pointers
 - Pointers and Functions
- **Arrays**
- Strings
- C Memory model
- Stack
- Heap

C Arrays

An array is just a block of variables of the same type, in consecutive memory addresses.

They act very similarly to pointers.

```
1  int arr[2];           // Declares a 2-element array
2
3  int arr_filled[] = {795, 635}; // Declares and fills an array
4
5  int x = arr_filled[0]; // Accesses elements of arrays
```

Arrays are (sort of) pointers

Arrays have an... interesting relationship with pointers

```
uint32_t arr[] = {50, 60, 70};
```

```
uint32_t *q = arr;  
uint32_t *r = &arr;  
uint32_t x = *arr;  
uint32_t y = sizeof arr;  
uint32_t z = sizeof(q);
```

		+0	+1	+2	+3
uint32_t arr	0x100		50		
	0x104		60		
	0x108		70		
uint32_t *q	0x10C		???????		
uint32_t *r	0x110		???????		
uint32_t x	0x114		???????		
uint32_t y	0x118		???????		
uint32_t z	0x11C		???????		
...					

Arrays are (sort of) pointers

In most circumstances, if an array is used as a variable, it is interpreted as a pointer to the first element.

```
uint32_t arr[] = {50,60,70};  
uint32_t *q = arr;  
uint32_t *r = &arr;  
uint32_t x = *arr;  
uint32_t y = sizeof arr;  
uint32_t z = sizeof(q);
```

		+0	+1	+2	+3
uint32_t arr	0x100		50		
	0x104		60		
	0x108		70		
uint32_t *q	0x10C	0x0000 0100			
uint32_t *r	0x110	????????			
uint32_t x	0x114	????????			
uint32_t y	0x118	????????			
uint32_t z	0x11C	????????			
...					

Arrays are (sort of) pointers

But when used with the address operator, it turns into a pointer to the whole array (which is the same numerical value as arr)

```
uint32_t arr[] = {50,60,70};  
uint32_t *q = arr;  
uint32_t *r = &arr;  
uint32_t x = *arr;  
uint32_t y = sizeof arr;  
uint32_t z = sizeof(q);
```

		+0	+1	+2	+3
uint32_t arr	0x100	50			
	0x104	60			
	0x108	70			
uint32_t *q	0x10C	0x0000	0100		
uint32_t *r	0x110	0x0000	0100		
uint32_t x	0x114	???????			
uint32_t y	0x118	???????			
uint32_t z	0x11C	???????			
...					

Arrays are (sort of) pointers

Since `arr` is a pointer to the first element, `*arr` is just the first element of the array

```
uint32_t arr[] = {50, 60, 70};  
uint32_t *q = arr;  
uint32_t *r = &arr;  
uint32_t x = *arr;  
uint32_t y = sizeof arr;  
uint32_t z = sizeof(q);
```

		+0	+1	+2	+3
uint32_t arr	0x100		50		
	0x104		60		
	0x108		70		
uint32_t *q	0x10C	0x0000 0100			
uint32_t *r	0x110	0x0000 0100			
uint32_t x	0x114		50		
uint32_t y	0x118	????????			
uint32_t z	0x11C	????????			
...					

Arrays are (sort of) pointers

The size of arr is the size of the array as a whole, which is 12 bytes (note that sizeof doesn't need parentheses)

```
uint32_t arr[] = {50,60,70};  
uint32_t *q = arr;  
uint32_t *r = &arr;  
uint32_t x = *arr;  
uint32_t y = sizeof arr;  
uint32_t z = sizeof(q);
```

		+0	+1	+2	+3
uint32_t arr	0x100		50		
	0x104		60		
	0x108		70		
uint32_t *q	0x10C	0x0000 0100			
uint32_t *r	0x110	0x0000 0100			
uint32_t x	0x114		50		
uint32_t y	0x118		12		
uint32_t z	0x11C	????????			
...					

Arrays are (sort of) pointers

When we try to get the size of `q`,
we get the size of one pointer,
which is 4 bytes

```
uint32_t arr[] = {50,60,70};  
uint32_t *q = arr;  
uint32_t *r = &arr;  
uint32_t x = *arr;  
uint32_t y = sizeof arr;  
uint32_t z = sizeof(q);
```

		+0	+1	+2	+3
uint32_t arr	0x100		50		
	0x104		60		
	0x108		70		
uint32_t *q	0x10C	0x0000 0100			
uint32_t *r	0x110	0x0000 0100			
uint32_t x	0x114		50		
uint32_t y	0x118		12		
uint32_t z	0x11C		4		
...					

Pointer Arithmetic (Pointers are sort of arrays)

Pointers are just numbers, so you can add and subtract to them (though C adds some complications).

When you add n to an int pointer, C adds $n * \text{sizeof(int)}$ instead of just n .

```
uint32_t arr[] = {50, 60, 70};
```

```
uint32_t *q = arr;
```

```
uint32_t *r = q+1;
```

```
uint32_t x = *(arr+2);
```

```
uint32_t y = arr[1];
```

```
uint32_t z = 3[arr];
```

		+0	+1	+2	+3
uint32_t arr	0x100		50		
	0x104		60		
	0x108		70		
uint32_t *q	0x10C	0x0000 0100			
uint32_t *r	0x110	????????			
uint32_t x	0x114	????????			
uint32_t y	0x118	????????			
uint32_t z	0x11C	????????			
...					

Pointer Arithmetic (Pointers are sort of arrays)

```
q = 0x0000 0100
q+1 -> 0x0000 0100 + 1*sizeof(int)
q+1 -> 0x0000 0100 + 4
0x0000 0104
```

```
uint32_t arr[] = {50,60,70};
uint32_t *q = arr;
uint32_t *r = q+1;
uint32_t x = *(arr+2);
uint32_t y = arr[1];
uint32_t z = 3[arr];
```

		+0	+1	+2	+3
uint32_t arr	0x100		50		
	0x104		60		
	0x108		70		
uint32_t *q	0x10C	0x0000 0100			
uint32_t *r	0x110	0x0000 0104			
uint32_t x	0x114	????????			
uint32_t y	0x118	????????			
uint32_t z	0x11C	????????			
...					

Pointer Arithmetic (Pointers are sort of arrays)

```
arr = 0x0000 0100
arr+2 -> 0x0000 0100 + 2*sizeof(int)
arr+2 = 0x0000 0108
*(arr+2) = 70
```

```
uint32_t arr[] = {50,60,70};
uint32_t *q = arr;
uint32_t *r = q+1;
uint32_t x = *(arr+2);
uint32_t y = arr[1];
uint32_t z = 3[arr];
```

		+0	+1	+2	+3
uint32_t arr	0x100		50		
	0x104		60		
	0x108		70		
uint32_t *q	0x10C	0x0000 0100			
uint32_t *r	0x110	0x0000 0104			
uint32_t x	0x114		70		
uint32_t y	0x118	????????			
uint32_t z	0x11C	????????			
...					

Pointer Arithmetic (Pointers are sort of arrays)

`arr[1]` is actually just shorthand for `*(arr+1)`. Because of how C defines pointer arithmetic, it works the same way as your usual array access.

```
uint32_t arr[] = {50,60,70};  
uint32_t *q = arr;  
uint32_t *r = q+1;  
uint32_t x = *(arr+2);  
uint32_t y = arr[1];  
uint32_t z = 3[arr];
```

		+0	+1	+2	+3
uint32_t arr	0x100		50		
	0x104		60		
	0x108		70		
uint32_t *q	0x10C	0x0000 0100			
uint32_t *r	0x110	0x0000 0104			
uint32_t x	0x114		70		
uint32_t y	0x118		60		
uint32_t z	0x11C	????????			
...					

Pointer Arithmetic (Pointers are sort of arrays)

```
3[arr] -> *(3+arr)
3+arr = 3*sizeof(int)+0x100
3+arr = 0x10C
*(3+arr) = 0x0000 0100
z = 256
```

```
uint32_t arr[] = {50,60,70};
uint32_t *q = arr;
uint32_t *r = q+1;
uint32_t x = *(arr+2);
uint32_t y = arr[1];
uint32_t z = 3[arr];
```

		+0	+1	+2	+3
uint32_t arr	0x100		50		
	0x104		60		
	0x108		70		
uint32_t *q	0x10C	0x0000 0100			
uint32_t *r	0x110	0x0000 0104			
uint32_t x	0x114		70		
uint32_t y	0x118		60		
uint32_t z	0x11C		256		
...					

Pointer Arithmetic (Pointers are sort of arrays)

DON'T WRITE `3[arr]`

Array accesses are just pointer arithmetic in disguise

Pointers can be used instead of arrays in many cases

```
uint32_t arr[] = {50,60,70};
```

```
uint32_t *q = arr;
```

```
uint32_t *r = q+1;
```

```
uint32_t x = *(arr+2);
```

```
uint32_t y = arr[1];
```

```
uint32_t z = 3[arr];
```

		+0	+1	+2	+3
uint32_t arr	0x100		50		
	0x104		60		
	0x108		70		
uint32_t *q	0x10C	0x0000 0100			
uint32_t *r	0x110	0x0000 0104			
uint32_t x	0x114		70		
uint32_t y	0x118		60		
uint32_t z	0x11C		256		
...					

Arrays are primitive

Arrays won't catch if you go out of bounds

If you're not careful, you can change other entirely unrelated variables

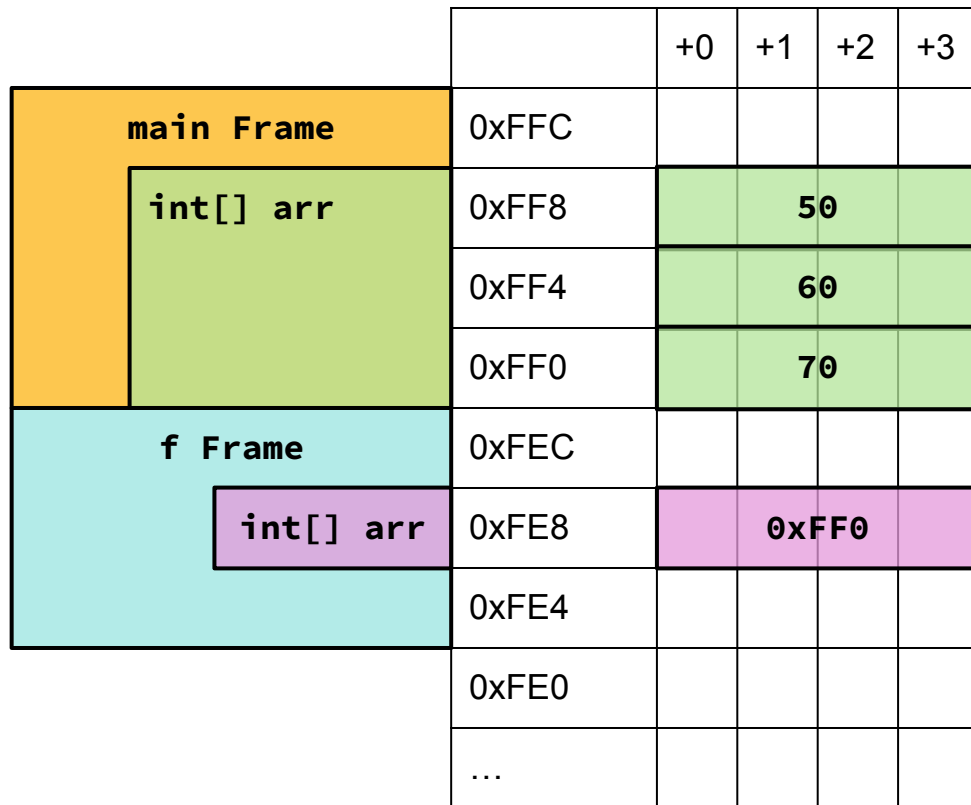
```
uint32_t arr[] = {50,60,70};  
uint32_t *q = arr;  
uint32_t *r = q+1;  
uint32_t x = *(arr+2);  
uint32_t y = arr[1];  
uint32_t z = 3[arr];  
arr[5] = 1000;
```

		+0	+1	+2	+3
uint32_t arr	0x100		50		
	0x104		60		
	0x108		70		
uint32_t *q	0x10C	0x0000 0100			
uint32_t *r	0x110	0x0000 0104			
uint32_t x	0x114		1000		
uint32_t y	0x118		60		
uint32_t z	0x11C		256		
...					

Arrays are primitive

Arrays will turn into pointers when passed as function inputs. **This loses the size of the array.**

```
void foo(int[] arr) {  
    ...  
}  
int[] arr = {1,2,3};  
foo(arr);
```



Arrays are primitive

Arrays will turn into pointers when passed as function inputs.
When sending in an array input, always include a size as input, and make sure they match.

```
void foo(int[] arr, int  
size) {  
    ...  
}  
int[] arr = {1,2,3};  
foo(arr,3);
```

		+0	+1	+2	+3
main Frame		0xFFC			
	int[] arr	0xFF8		50	
		0xFF4		60	
		0xFF0		70	
f Frame		0xFEC			
	int[] arr	0xFE8		0xFF0	
	int size	0xFE4		3	
		0xFE0			
	...				

Arrays are primitive

Arrays are local variables. When you return from a function, all local variables get deleted*.

```
int* foo() {  
    int[] arr = {1,2,3};  
    return arr;  
}  
int* p = foo();  
printf("%d\n", p[2]);  
printf("%d\n", p[2]);
```

		+0	+1	+2	+3
main Frame	0xFFC				
	int* p	0xFF8	???????		
		0xFF4			
f Frame	0xFF0				
	int[] arr	0xFEC	3		
		0xFE8	2		
		0xFE4	1		
		0xFE0			
	...				

Arrays are primitive

(It takes time to delete something, so we just leave the data as is and let anyone else overwrite it)

```
int* foo() {  
    int[] arr = {1,2,3};  
    return arr;  
}  
int* p = foo();  
printf("%d\n", p[2]);  
printf("%d\n", p[2]);
```

		+0	+1	+2	+3
main Frame int* p	0xFFC				
	0xFF8		0xFE4		
	0xFF4				
	0xFF0				
	0xFEC		3		
	0xFE8		2		
	0xFE4		1		
	0xFE0				
	...				

Arrays are primitive

This print statement happens to work, because we get the data before the printf frame is fully formed

```
int* foo() {  
    int[] arr = {1,2,3};  
    return arr;  
}  
int* p = foo();  
printf("%d\n", p[2]);  
printf("%d\n", p[2]);
```

		+0	+1	+2	+3
main Frame		0xFFC			
	int* p	0xFF8	0xFE4		
		0xFF4			
printf Frame		0xFF0			
	char* format	0xFEC	0x1000 8000		
	int arg	0xFE8	3		
		0xFE4			
		0xFE0			
	...				

Arrays are primitive

After the print statement finishes, its frame is also freed.

```
int* foo() {  
    int[] arr = {1,2,3};  
    return arr;  
}  
int* p = foo();  
printf("%d\n", p[2]);  
printf("%d\n", p[2]);
```

		+0	+1	+2	+3
main Frame int* p	0xFFC				
	0xFF8		0xFE4		
	0xFF4				
	0xFF0				
	0xFEC	0x1000 8000			
	0xFE8		3		
	0xFE4				
	0xFE0				
	...				

Arrays are primitive

The second time you try to print the same thing, you get a different result.

```
int* foo() {  
    int[] arr = {1,2,3};  
    return arr;  
}  
int* p = foo();  
printf("%d\n", p[2]);  
printf("%d\n", p[2]);
```

		+0	+1	+2	+3
main Frame		0xFFC			
	int* p	0xFF8	0xFE4		
		0xFF4			
printf Frame		0xFF0			
	char* format	0xFEC	0x1000 8000		
	int arg	0xFE8	0x1000 8000		
		0xFE4			
		0xFE0			
	...				

Arrays are primitive, Summary

1. Array bounds are not checked during element access.
2. An array is passed to a function as a pointer.
3. Declared arrays are only allocated while the scope is valid.

Consequences? [Segmentation faults](#), [bus errors](#), ...

- These are VERY difficult to find; be careful!
- You'll learn how to debug these in Lab with gdb...

Agenda

- Pointers
 - Pointers and Functions
- Arrays
- **Strings**
- C Memory model
- Stack
- Heap

ASCII

- Unlike with numbers, there's no “natural” way to assign bitstrings to characters
- Current standard: ASCII
- Each character uses 8 bits of data (one byte)
- C does not distinguish between chars and 8-bit integers
 - Ex. The character '2' corresponds to ASCII value 50. If we did '2' + '2', C would interpret this as $50+50 = 100$, which, if turned back into a char, would be the character 'b'.
- The next slide will have a shortened version of the ASCII table; this will be provided whenever ASCII translations are needed.

ASCII table

HEX	DEC	CHAR	HEX	DEC	CHAR	HEX	DEC	CHAR	HEX	DEC	CHAR	HEX	DEC	CHAR	HEX	DEC	CHAR
0x20	32	SPACE	0x30	48	0	0x40	64	@	0x50	80	P	0x60	96	`	0x70	112	p
0x21	33	!	0x31	49	1	0x41	65	A	0x51	81	Q	0x61	97	a	0x71	113	q
0x22	34	"	0x32	50	2	0x42	66	B	0x52	82	R	0x62	98	b	0x72	114	r
0x23	35	#	0x33	51	3	0x43	67	C	0x53	83	S	0x63	99	c	0x73	115	s
0x24	36	\$	0x34	52	4	0x44	68	D	0x54	84	T	0x64	100	d	0x74	116	t
0x25	37	%	0x35	53	5	0x45	69	E	0x55	85	U	0x65	101	e	0x75	117	u
0x26	38	&	0x36	54	6	0x46	70	F	0x56	86	V	0x66	102	f	0x76	118	v
0x27	39	'	0x37	55	7	0x47	71	G	0x57	87	W	0x67	103	g	0x77	119	w
0x28	40	(	0x38	56	8	0x48	72	H	0x58	88	X	0x68	104	h	0x78	120	x
0x29	41	)	0x39	57	9	0x49	73	I	0x59	89	Y	0x69	105	i	0x79	121	y
0x2A	42	*	0x3A	58	:	0x4A	74	J	0x5A	90	Z	0x6A	106	j	0x7A	122	z
0x2B	43	+	0x3B	59	;	0x4B	75	K	0x5B	91	[	0x6B	107	k	0x7B	123	{
0x2C	44	,	0x3C	60	<	0x4C	76	L	0x5C	92	\	0x6C	108	l	0x7C	124	
0x2D	45	-	0x3D	61	=	0x4D	77	M	0x5D	93	]	0x6D	109	m	0x7D	125	}
0x2E	46	.	0x3E	62	>	0x4E	78	N	0x5E	94	^	0x6E	110	n	0x7E	126	~
0x2F	47	/	0x3F	63	?	0x4F	79	O	0x5F	95		0x6F	111	o	0x00	0	NULL

Strings

- Strings in C are defined as arrays of characters, similar to Python.
 - There isn't a primitive type called "string" in C. C strings are "char[]"s or "char*"s.
- Problem: C arrays don't know their own length (especially as pointers).
 - If we want to print out a string, we don't know when to stop printing...
- How to fix?
 - Approach 1: Store a length attribute (like Java does)
 - Approach 2 (what C does): Add a special "END" character to the end of the string
- For C, we use byte 0 ('\0', NULL) to represent the end of a string. This is known as the **null terminator**.

Strings

- For C, we use byte 0 ('\0', NULL) to represent the end of a string. This is known as the null terminator.
- Ex. The string “Hi” is stored in memory as an array of characters. This array would look like: { 'H', 'i', '\0' }, or in number form, {72, 105, 0}
- Strings need one more byte of memory than their length to be stored.
- Any string literals you use in your code will add a null terminator for you.
- However, if you create a string character-by-character in an array, you need to remember to add the null terminator yourself. **If the null terminator is missing, C will keep reading.**

Agenda

- Pointers
 - Pointers and Functions
- Arrays
- Strings
- **C Memory model**
- Stack
- Heap

0xFFFF...FFF

Summer 2026

-
- The diagram illustrates the memory layout of a program, divided into four main segments:
- Stack:** Located at the top of the memory, it grows downwards. It is represented by a series of horizontal bars.
 - Heap:** Located below the Stack, it grows upwards. It is represented by a series of horizontal bars. Three blue dots are shown in the middle of the Heap segment, indicating memory allocation.
 - Static/data:** Located below the Heap, it contains static and global variables. It is represented by a series of horizontal bars.
 - Code/text:** Located at the bottom of the memory, it contains the executable code. It is represented by a series of horizontal bars.
- The memory address `0x0000...000` is shown at the bottom left, indicating the starting address of the Code/text segment.

0x0000...000

Memory Structure

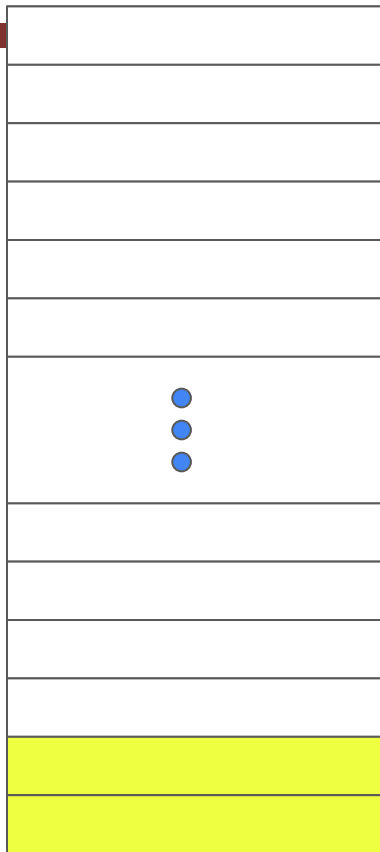
0xFFFF...FF

CS 61C

Summer 2026

Code/text

- Executables are data too!
 - Big improvement over early computers (when programs were physical wire configurations)
- The code segment stores the actual bytecode that comprises your program.
- Fixed size, ideally never changed after loading the program
- Includes some constants!
 - Constants that are considered "built-in" to the code
 - Ex. $x = y + 1$; the 1 is part of the code.



0x0000...000

Memory Structure

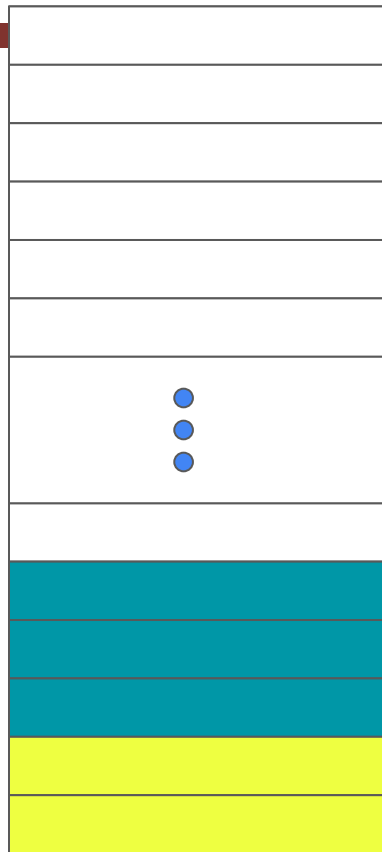
0xFFFF...FFF

CS 61C

Summer 2026

Static/data

- Stores anything that's a fixed size:
 - Global variables (you only ever need one copy of a global variable, so fixed size)
 - String literals (you can determine how many string literals there are in a program)



0x0000...000

Memory Structure

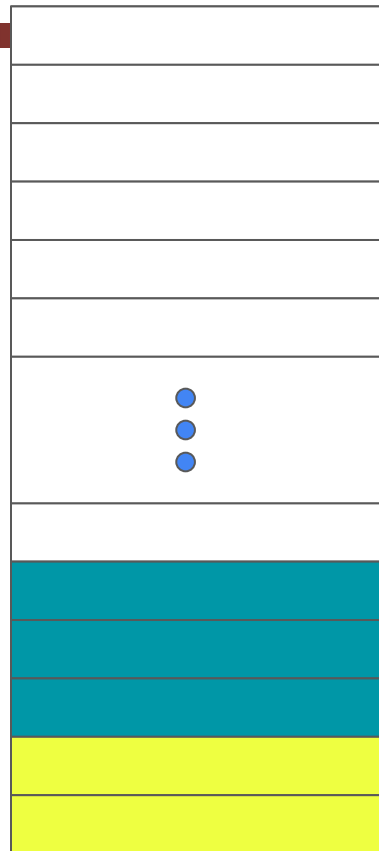
0xFFFF...FFF

CS 61C

Summer 2026

Static/data

- Every unique string gets stored once.
- If you have the same string several times in the code, only one copy gets saved (everything points to that copy).
- Corollary: String literals are **immutable** if they are stored in data.
 - `char* i = "Hi";`
 `i[0]="P"; // Error`
 `char* j = "Hi";`
- Only applies to string literals which need pointers!
- Ex. `char[] i = "Hi";` doesn't need a pointer to "Hi", so it treats the string the same as an integer literal.



0x0000...000

Agenda

- Pointers
 - Pointers and Functions
- Arrays
- Strings
- C Memory model
- **Stack**
- Heap

Memory Structure

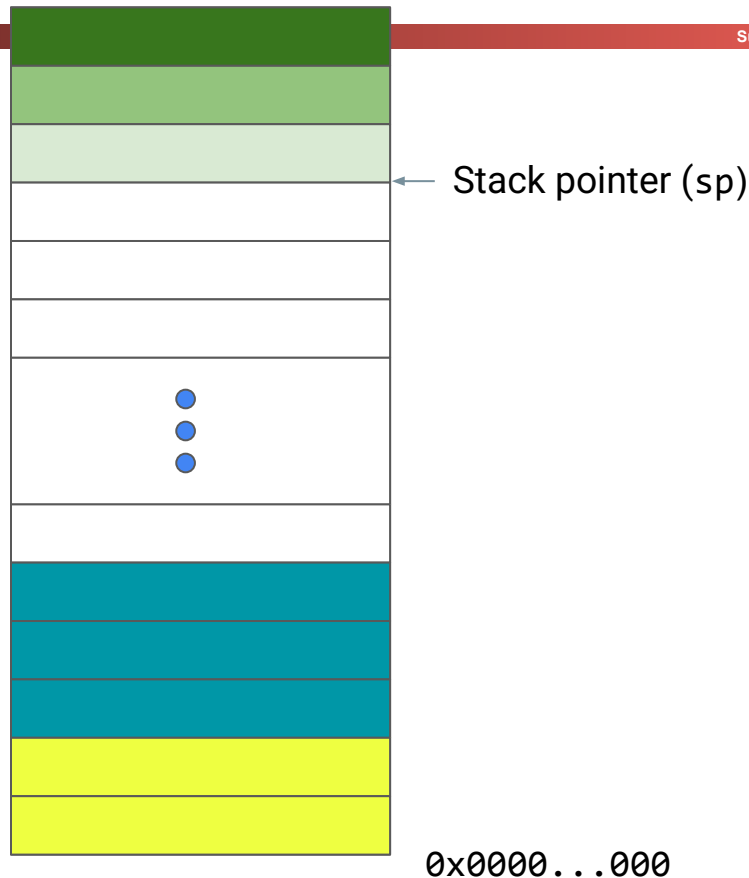
0xFFFF...FFF

CS 61C

Summer 2026

Stack

- Acts similarly to 61A "Environment Diagrams"
- Every function call sets aside some space on the stack for its local variables (plus some metadata)
- Designed for **temporary** storage; after a function returns, all data in the stack frame gets freed.
- Variable size; grows downward as you call functions and shrinks as you return from functions.
- All local variables are defined on the stack

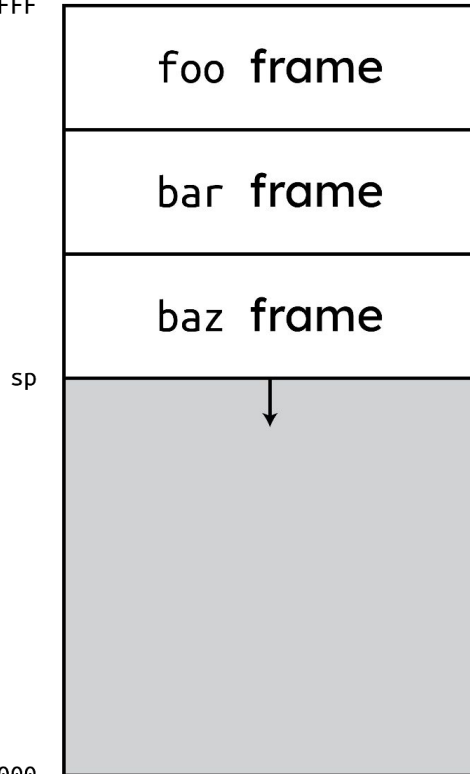


Stack Frames

- Stack frames form a contiguous block of memory
- The bottom of the stack is marked by a **stack pointer**
- Example:

```
foo() { bar(); }  
bar() { baz(); }  
baz() { ... }
```

0xFFFF FFFF

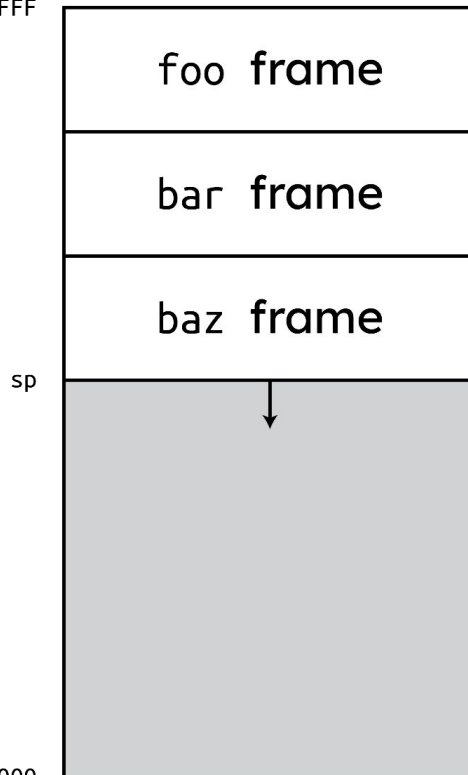


0x0000 0000

Common C Bug: Passing Stack Pointers

- What if bar returns a pointer to its local variable?
- What if foo gives bar a pointer to its local variable?
- Key idea:
 - Passing pointers into the stack is fine in one direction (caller to callee)
 - And dangerous in the other (callee to caller)

0xFFFF FFFF



0x0000 0000

Agenda

- Pointers
 - Pointers and Functions
- Arrays
- Strings
- C Memory model
- Stack
- **Heap**

Memory Structure

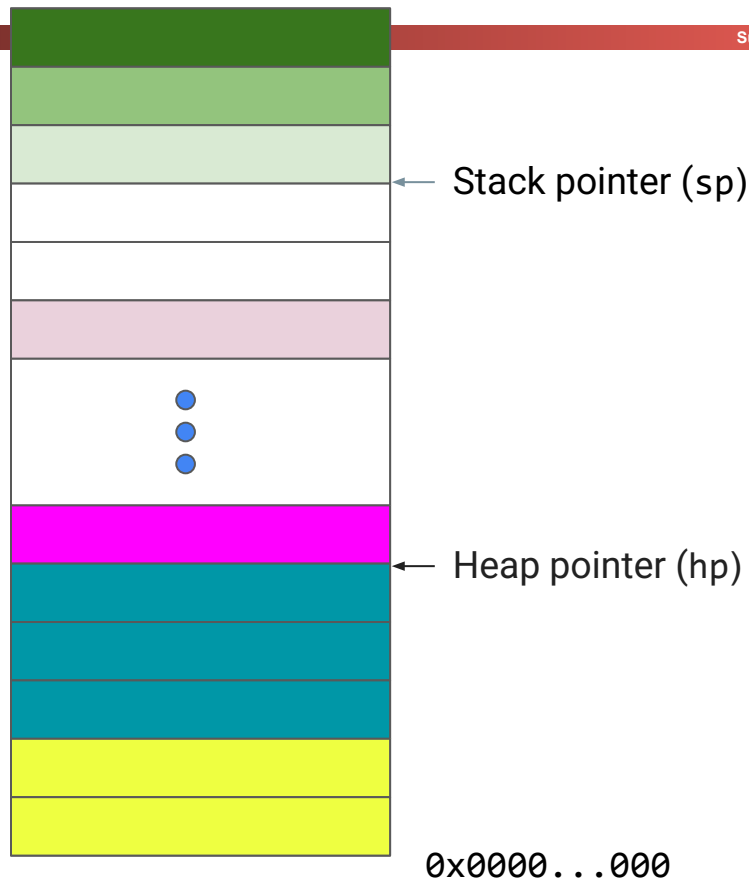
0xFFFF...FFF

CS 61C

Summer 2026

Heap

- Stack variables disappear when the function returns
- For any variables that need to persist across functions, use the heap
- In C, all heap memory must be manually allocated.
- Similarly, heap memory is only freed manually. **No garbage collector.**
- Grows upward as memory is allocated, and shrinks as memory is freed
- Big source of bugs



Heap

- Has nothing to do with the data structure
- The heap is the most flexible part of memory, but it's also the part that causes the most problems
 - Used similarly to Java's `new` command, but you need to do everything manually.
- Huge pool of memory (usually >> stack) **not** allocated in contiguous order.
- The heap is run by a *memory manager*, which is part of the *operating system* (or OS). If you want to use the heap, you need to ask for permission from the manager.

malloc

- `void* malloc(size_t n)`
 - Requests `n` consecutive bytes from the memory manager. The memory manager finds a block of open space and reserves that space for your personal use. This is known as a *Memory ALLOCation*.
 - The manager doesn't erase anything in that space; it's your responsibility to take out the garbage data
 - If the memory manager can't find enough memory (or you've used too much memory), returns a NULL pointer
 - Until you return the memory, the memory manager won't give that space to anyone else. It's yours.
- Analogy: the memory manager is a storage center. When you malloc, you ask the manager for a space, and they give you a key to a space that's big enough for you (but they don't clean that space out)

Heap Usage

- `void free(void* ptr)`
 - Gives the block pointed to by `ptr` back to the memory manager
 - `ptr` MUST be a pointer that was returned by the memory manager earlier that hasn't been freed; if you free a pointer to the stack, a pointer in the middle of an allocated block, or a pointer that was already freed, your code crashes
 - Does NOT clean out any data you put in the allocated block
- Analogy: Once you're done with that space, you return the key to the manager so they can give that space to someone else

Heap Usage

- `void* calloc(size_t nitems, size_t size)`
 - Allocates (`nitems * size`) bytes of data, and guarantees that the returned block is cleared (Clear ALLOCation)
 - Two main ways the memory manager does this:
 - Find a block that's already empty
 - Run `malloc`, then clear the block by zeroing out its contents
 - Either way, this makes `calloc` slightly slower than `malloc`
- Analogy: Same thing as `malloc`, but you pay a little extra to have the manager give you a storage space that's already empty

Heap Usage

- `void* realloc(void* ptr, size_t n)`
 - "Resizes" the block pointed to by `ptr`, so that it now contains `n` bytes.
 - `ptr` must be a pointer that was returned by the memory manager
 - Most of the time, it just resizes the box and returns the same pointer back. However, if it can't do so (ex. if the data immediately after the malloc'ed block is already allocated), the memory manager can choose to move the block to a new place in memory, freeing the old pointer.
 - Otherwise, acts as `malloc`; newly allocated memory contains garbage, and `realloc` returns `NULL` if it can't find a new block.
- Analogy: You need a larger-size storage space, so you give your current key back and get a new key to a bigger space. The manager moves your data to the new space for you. Alternatively, they tear down a wall and give you back your old key.

Heap Example

```
int* i = malloc(sizeof(*i) * 5);
for(int j = 0; j < 5; j++) i[j] = j;
int* k = calloc(3, sizeof(*k));
i = realloc(i, sizeof(*i)*2);
i = realloc(i, sizeof(*i)*10);
free(k);
k = malloc(sizeof(*k)*6);
for(int j = 0; j < 5; j++) k[j] = j;
free(i);
free(k+2);
```

Best Practices for Heap Use

- Return your memory before you forget
 - If you don't free a pointer before you lose track of the pointer, it'll stay allocated forever. This is known as a *memory leak*, and results in code that eventually breaks.
 - Frees are NOT recursive; if you have a complicated struct, free all components of the struct BEFORE you free the struct itself
- Use `sizeof` when deciding how much memory to allocate
 - You can use `sizeof(int)` in the previous example... but it makes the code less flexible if you later change `int` to something else.
- In most cases, the stack is faster to use (no need to search for memory), but also can't store as much data and can't store it long. Use the stack when you want something fast, and the heap when you want something last.
- Tomorrow: More ways to break the heap